9 File, Argomenti da Riga di Comando e Scope

Questa dispensa approfondisce tre aspetti fondamentali del linguaggio C: **gestione dei file**, **acquisizione degli argomenti da riga di comando** e **regole dello scope delle variabili**. Si tratta di strumenti fondamentali per rendere i programmi più flessibili e in grado di interagire con l'esterno.

9.1 I File in C

Un **file** è una struttura dati persistente memorizzata su memoria di massa. A differenza delle variabili in memoria, i file mantengono i dati anche dopo la fine dell'esecuzione del programma.

Per manipolare un file in C si utilizza una variabile puntatore di tipo `FILE*` che rappresenta il **descrittore del file**.

9.1.1 Apertura di un File

```
FILE *fp;
fp = fopen("dati.txt", "r");  // apertura in lettura
```

La funzione `fopen` restituisce:

- un puntatore a `FILE` (inizializzato all'inizio del file) se l'apertura ha successo.
- `NULL` in caso di errore.

Modalità di apertura più comuni

- `"r"` lettura (testo)
- `"w"` scrittura (crea o sovrascrive)
- `"a"` append

- `"rb"`, `"wb"`, `"ab"` versioni binarie

Dopo l'apertura, il programma opera sul file utilizzando la variabile che lo rappresenta. Al termine, la corrispondenza dovrà essere eliminata: operazione di *chiusura* del file.

9.1.2 Chiusura di un File

```
fclose(fp);
```

Il valore restituto è un intero:

- 0 se la chiusura ha successo.
- EOF (valore intero negativo) in caso di errore.

Inoltre, l'utilizzo della funzione `fclose` assicura che tutti i dati eventualmente rimasti in buffer vengano scritti su disco.

9.1.3 Lettura e Scrittura di un File

Un file di testo è un file che contiene sequenze di caratteri. Per questo motivo un file possiede delle caratterische specifiche che lo distinguono da un file binario:

- esiste un concetto di riga e di fine riga ('\n').
- i dati sono rappresentati come sequenze di caratteri ASCII.
- la sequenza di caratteri è terminata dal carattere speciale EOF.

I canali di I/O standard non sono altro che file di testo speciali:

- `stdin` (standard input): tastiera
- `stdout` (standard output): schermo
- `stderr` (standard error): schermo (per messaggi di errore)

Lettura e Scrittura Formattata Per leggere o scrivere in un file di testo in maniera formattata ed effettuando le conversioni richieste dove necessario si usano `fscanf` e `fprintf`.

```
fprintf(fp, "%d %c", numero, carattere);
fscanf(fp, "%d %c", &numero, &carattere);
```

La lettura restituisce un intero che indica il numero di *campi* letti con successo. La scrittura restituisce il numero di *caratteri* scritti con successo.

Lettura Carattere per Carattere

```
int c;
c = fgetc(fp);
while (c != EOF) {
    putchar(c);
    c = fgetc(fp);
}
```

Lettura di Linee di Testo

```
char buffer[100];
while (fgets(buffer, 100, fp) != NULL) {
    fputs(buffer, stdout);
}
```

9.1.4 File Binari: fread e fwrite

Un file binario è una sequenza di byte senza formattazione specifica. Può essere usato per archiviare qualunque tipo di informazione. In questo caso input e output avvengono sotto forma di una sequenza di byte.

Rispetto ai file di testo, con i file binari la fine del file non è segnata da un carattere speciale (EOF), ma si rileva in base all'esito delle operazioni di lettura.

```
// legge n sequenze di byte di lunghezza Dato
fread(indirizzo, sizeof(Dato), n, fp);

// scrive n sequenze di byte di lunghezza Dato
fwrite(indirizzo, sizeof(Dato), n, fp);
```

la funzione `fwrite` restituisce il numero di elementi scritti con successo (che possono essere meno di n), prelevando i byte a partire dall'indirizzo specificato. Analogamente, `fread` restituisce il numero di elementi letti con successo, che possono essere meno di n se si raggiunge la fine del file. Controllare il valore restituito è il solo modo per sapere cosa è stato letto, in particolare per capire se il file è terminato.

9.1.5 Accesso Diretto: fseek e ftell

```
fseek(fp, 0L, SEEK_END);    // va alla fine del file
pos = ftell(fp);            // posizione corrente
rewind(fp);                 // equivalente a fseek(fp,0,SEEK_SET)
```

Nota: La funzione `fseek` permette di spostare il puntatore di lettura/scrittura in una posizione arbitraria del file. La funzione fornisce tre punti di partenza per lo spostamento:

- `SEEK_SET`: dall'inizio del file.
- `SEEK_CUR`: dalla posizione corrente.
- `SEEK_END`: dalla fine del file.

9.2 Argomenti da Riga di Comando

È possibile passare informazioni a un programma direttamente da terminale:

```
./programma arg1 arg2 arg3
```

La funzione `main` può ricevere due parametri:

```
int main(int argc, char *argv[]) {
    ...
}
```

- `argc` = numero totale di argomenti (il primo è il nome del programma);
- `argv` = array di stringhe con gli argomenti.

Esempio

```
./programma prova.txt
```

```
int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Uso: %s nomefile\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "r");
    if (fp == NULL) {
        printf("Errore apertura file!\n");
        return 1;
    }

    /* elaborazione */
    fclose(fp);
    return 0;
}
```

Conversione di argomenti numerici

```
int n = atoi(argv[1]);
float x = atof(argv[2]);
char c = argv[3][0];
```

9.3 Variabili Locali, Statiche e Globali

Il linguaggio C distingue diverse categorie in base allo *tempo di vita* e allo *scope*.

9.3.1 Variabili Locali Automatiche

Vivono solo all'interno della funzione in cui sono dichiarate. Sono allocate sullo stack.

```
void f() {
    int x = 0; // nuova x a ogni chiamata
}
```

9.3.2 Variabili Locali Statiche

Mantengono il proprio valore anche dopo la fine della funzione.

```
void f() {
    static int x = 0;
    x++;
    printf("%d\n", x);
}
```

Ogni chiamata stampa un valore crescente.

9.3.3 Variabili Globali

Sono dichiarate fuori da qualsiasi funzione. Sono visibili da tutto il file (scope di file).

```
int contatore = 0; // globale

void f() { contatore++; }
```

Attenzione L'uso indiscriminato di variabili globali rende difficile il debug e la manutenibilità del codice.

9.3.4 Blocchi di Istruzioni e Regole dello Scope

Un **blocco** è un insieme di istruzioni racchiuse tra `{` e `}`.

Le variabili dichiarate in un blocco esistono solo all'interno di quel blocco.

```
int x = 1;
if (x > 0) {
    int x = 5;   // nasconde la x esterna
    printf("%d\n", x); // stampa 5
}
printf("%d\n", x);      // stampa 1
```

Una dichiarazione interna con lo stesso nome di una variabile esterna **nasconde** temporaneamente quella esterna.

9.4 Esercizi

Esercizio 9.1

Si scriva un programma che, aperto un file di testo di nome "output.txt", in modalità scrittura, provveda a leggere da input delle parole separate da spazi (stringhe di al più 63 caratteri) e le scriva nel file di testo. La lettura termina quando l'utente inserisce la parola "fine". Successivamente, il programma riapre il file in modalità lettura e stampa a video tutte le parole lette dal file.

Esercizio 9.2

Si scriva un programma che apre un file di testo il cui nome è input1.txt contenente una sequenza di lunghezza indefinita (0 o più) di numeri interi. Il programma individua nel file il valore massimo e lo scrive in un altro file output1.txt insieme al numero di volte che vi compare.

Esercizio 9.3

Una matrice sparsa è una matrice in cui la maggior parte degli elementi è costituita da zeri. Spesso una matrice sparsa viene rappresentata in formato COO (Coordinate List), ovvero come una sequenza di triple (i, j, valore) che indicano che, nella posizione (i, j), è presente un valore diverso da zero (valore $\neq 0$). Si consideri un file binario denominato "input6.bin", che contiene due interi rappresentanti il numero di righe e di colonne della matrice, seguiti dalla matrice stessa rappresentata in modalità ordinaria (cioè come sequenza di r * c elementi, dove r è il numero di righe e c il numero di colonne), i cui elementi sono numeri in virgola mobile a doppia precisione. Si scriva un

programma che legga tale file e salvi in un secondo file di testo denominato "coo6.txt" la rappresentazione COO della matrice.

Esercizio 9.4

Dato un file di testo "dati.txt" contenente i dati relativi agli studenti immatricolati al primo anno della Facoltà di Ingegneria. In particolare, le informazioni sono memorizzate nel file con uno studente per riga nel seguente formato:

- matricola (intero);
- corso di laurea (intero);

Sia dato un secondo file di testo, "indirizzi.txt" che contiene l'indirizzo di ogni studente. In particolare, in ogni linea, separati da uno spazio:

- matricola (intero);
- nome (stringa di al più 20 caratteri);
- cognome (stringa di al più 30 caratteri);
- via (stringa di al più 30 caratteri);
- città (stringa di al più 20 caratteri);
- cap (intero).

Si scriva un programma che:

1. a partire dai file "dati.txt" e "indirizzi.txt" costruisca una tabella contenente per ogni studente: matricola, nome, cognome, via, città, cap e corso di laurea.
2. a partire dalla tabella e dato input un intero C che rappresenta un corso di laurea, stampi la percentuale di studenti iscritti a quel corso.
3. scriva su un terzo file di testo "milano.txt", nome, cognome e numero di matricola di tutti gli studenti che abitano a Milano.

Esercizio 9.5

Una società di telefonia cellulare gestisce un programma di premiazione per "utenti fedeli". In particolare, per ogni cliente viene salvato su un file binario "punti.dat" il nome del cliente (al massimo 31 caratteri) e un numero intero che rappresenta i punti accumulati. Tali informazioni sono organizzate come una struttura `User`, opportunamente definita.

Si scriva una funzione `readPoints()` che ricevuto in ingresso il nome di un file `usersFile`, un array `results` di strutture `User`, la dimensione massima dell'array `maxDim`, e un limite

inferiore di punti `minPoints`, copi nell'array `results` i dati dei clienti che hanno almeno i punti specificati da `minPoints`.

La funzione deve restituire come risultato il numero di utenti con almeno `minPoints`. Si noti che tale risultato rappresenta anche la dimensione logica dell'array results. Qualora il file non sia accessibile, la funzione deve restituire il valore -1.

Si scriva poi un programma `main()` che chieda all'utente il numero di clienti salvati sul file (tale numero sarà noto solo a tempo di esecuzione), e allochi dinamicamente un vettore V di `User` sufficientemente grande per poter contenere, nel caso peggiore, i dati di tutti gli utenti salvati in `usersFile`. Il programma dovrà poi chiedere all'utente il minimo punteggio e, utilizzando la funzione `readPoints()`, leggere da file e memorizzare in V i dati degli utenti che hanno almeno il punteggio minimo specificato. Il programma infine deve stampare a video il nome ed il punteggio degli utenti contenuti in V se e solo se il nome comincia per "Me".

Esercizio 9.6

Si scriva un programma che decifra o cifra un file di testo applicando uno scorrimento ai caratteri alfabetici. Il programma deve accettare i parametri *in qualsiasi ordine* tramite dei flag da linea di comando:

- `-i <inputfile>` : specifica il file di input
- `-o <outputfile>` : specifica il file di output
- `-k <shift>` : specifica la chiave
- `-d` : decifra il file (se non presente, cifra)

Esempio:

```
# Cifra il file msg.txt con chiave 3 e salva il risultato in cif.txt
./programma -i msg.txt -o cif.txt -k 3

# Decifra il file cif.txt con chiave 5 e salva il risultato in msg.txt
./programma -k 5 -d -o msg.txt -i cif.txt
```

Esercizio 9.7

Si scriva un programma che gestisca un database di voti universitari salvati su un file "registro.txt". Il programma deve gestire due modilità operative distinte:

- Modalità aggiunta (-add): aggiunge un nuovo studente al registro. Richiede tre parametri: nome, cognome e voto.
- Modalità ricerca (-find): cerca e stampa a video tutti gli studenti che hanno un voto superiore o uguale ad una soglia specificata.

Si scriva una funzione `aggiungi_studente()` che aggiunge un nuovo studente al file "registro.txt" e una funzione `cerca_studenti()` che apre e cerca all'interno di "registro.txt" tutti gli studenti con voto superiore o uguale alla soglia specificata. Si gestiscano le due modalità come argomenti da riga di comando e si aggiunga la possibilità di utilizzare un file di registro personalizzato usando come flag "-f".

Esempio:

```
# Usa il file di default, aggiunge Mario Rossi con 28
./programma -add Mario Rossi 28

# Usa un file diverso, cerca voti >= 25
./programma -f esame_c.txt -find 25
```

Esercizio 9.8

Un file di testo "archivio.txt" contiene i dati (primo autore, titolo, numero di copie possedute, numero di copie in prestito) relativi ai differenti volumi conservati presso una biblioteca. Più precisamente, ogni riga del file contiene nell'ordine, separati da uno spazio bianco:

- autore (al più 23 caratteri senza spazi)
- titolo (al più 55 caratteri senza spazi)
- numero di copie possedute (intero)
- numero di copie in prestito (intero)

Si realizzi un programma C che:

1. Legga il contenuto di "archivio.txt" e costruisca in memoria un vettore V di strutture corrispondenti (si supponga che il file "archivio.txt" non possa contenere più di 30 righe). Si stampi a video il contenuto del vettore.

2. A partire da V, costruisca una lista L di interi contenente per ciascun volume il numero di copie disponibili nella biblioteca, ovvero la differenza fra il numero di copie possedute e il numero di copie in prestito. Si stampi a video il contenuto della lista L.

3. Utilizzando L per ottenere la somma delle copie disponibili e V per la somma delle copie possedute, calcoli il rapporto fra volumi disponibili e volumi posseduti.

4. Utilizzando la lista di interi L, stampi il numero di riga di "archivio.txt" relativo al volume con più copie disponibili. In caso di più volumi con pari numero di copie disponibili, qualunque riga relativa a questi ultimi è considerata una risposta corretta.

Soluzioni

Soluzione dell'esercizio 9.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DIM 64

int main() {
    FILE * fp;
    char s[DIM];

    if ((fp=fopen("output.txt", "w")) == NULL) {
        return -1;
    }

    int i = 0;
    do {

        // la scanf qui funziona come una gets perchè se scrivo "ciao mondo" in
            realtà
        // la scrittura reale avverrà in due cicli, nel primo scriverà "ciao" nel
            secondo
        // non aspetterà l'input dell'utente essendo già il buffer pieno e
            scriverà "mondo"

        scanf("%s", s); // non conta i \n ma solo gli spazi --> non va mai a capo
            su file

        // fgets(s, DIM, stdin); // conteggia i \n mandati da tastiera -->
            cambiare strcmp "fine\n"

        printf("%s\n", s);

        if (strcmp("fine", s) != 0)
            fprintf(fp, "%s ", s); // lo spazio nella "%s " stampa lo spazio dopo
                la prima serie di caratteri

        i++;
    } while (strcmp("fine", s) != 0);

    printf("Totale iterazioni %d\n", i);
    fclose(fp);

    fp = fopen("output.txt", "r");
    if (fp == NULL) {
        printf("Errore: Impossibile aprire il file per la lettura.\n");
        return 1;
    }

    /* fscanf funziona come scanf ma legge dal file.
       Restituisce EOF (End Of File) quando non c'è più nulla da leggere. */
    while (fscanf(fp, "%63s", s) != EOF) {
        printf("Parola letta: %s\n", s);
    }

    fclose(fp);
```

```
    return 0;
}
```

Soluzione dell'esercizio 9.2

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fin, *fout;
    int numero_letto;
    int max;
    int conteggio;
    int primo_numero; // Flag per il primo numero letto

    fin = fopen("input1.txt", "r");
    if (fin == NULL) {
        printf("Errore: Impossibile aprire input1.txt\n");
        return 1;
    }

    fout = fopen("output1.txt", "w");
    if (fout == NULL) {
        printf("Errore: Impossibile creare output1.txt\n");
        fclose(fin);
        return 1;
    }

    conteggio = 0;
    primo_numero = 1;

    while (fscanf(fin, "%d", &numero_letto) == 1) {

        if (primo_numero) {
            // Se è il primo numero, è automaticamente il massimo corrente
            max = numero_letto;
            conteggio = 1;
            primo_numero = 0; // Impostiamo il flag a falso
        }
        else {
            // Confronto con il massimo attuale
            if (numero_letto > max) {
                max = numero_letto;
                conteggio = 1; // Nuovo massimo trovato, resetto il conteggio a 1
            }
            else if (numero_letto == max) {
                conteggio++; // Stesso valore del massimo, incremento il conteggio
            }
        }
    }

    if (!primo_numero) {
        fprintf(fout, "%d %d\n", max, conteggio);
    } else {
        printf("Il file di input era vuoto.\n");
    }

    fclose(fin);
    fclose(fout);
```

```
    return 0;
}
```

Soluzione dell'esercizio 9.3

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fin, *fout;
    int righe, colonne;
    int i, j;
    double valore;
    int elementi_letti;

    // Apertura del file binario in lettura
    // "rb" è fondamentale per leggere correttamente i file binari
    fin = fopen("input6.bin", "rb");
    if (fin == NULL) {
        printf("Errore: Impossibile aprire input6.bin\n");
        return 1;
    }
    elementi_letti = fread(&righe, sizeof(int), 1, fin);
    elementi_letti += fread(&colonne, sizeof(int), 1, fin);

    if (elementi_letti != 2) {
        printf("Errore: Il file è troppo corto o corrotto.\n");
        fclose(fin);
        return 1;
    }

    fout = fopen("coo6.txt", "w");
    if (fout == NULL) {
        printf("Errore: Impossibile creare coo6.txt\n");
        fclose(fin);
        return 1;
    }

    for (i = 0; i < righe; i++) {
        for (j = 0; j < colonne; j++) {
            if (fread(&valore, sizeof(double), 1, fin) != 1) {
                printf("Attenzione: Fine del file inaspettata all'elemento (%d, %d
                    )\n", i, j);
                // Interrompiamo i cicli forzando gli indici al massimo
                i = righe;
                j = colonne;
            } else {
                // Se il valore è diverso da zero, salviamo la tripla (COO)
                if (valore != 0.0) {
                    // Scriviamo: indice riga, indice colonna, valore
                    fprintf(fout, "%d %d %lf\n", i, j, valore);
                }
            }
        }
    }

    fclose(fin);
    fclose(fout);

    printf("Conversione completata. Risultato salvato in coo6.txt\n");
```

```
    return 0;
}
```

Soluzione dell'esercizio 9.4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    unsigned int matr;
    unsigned int CDL;
} Dati;

typedef struct {
    unsigned int matr;
    char nome[21];
    char cognome[31];
    char via[31];
    char citta[31];
    unsigned int CAP;
} Indirizzo;

typedef struct {
    unsigned int matr;
    char nome[21];
    char cognome[31];
    char via[31];
    char citta[31];
    unsigned int CAP;
    unsigned int CDL;
} Elemento;

typedef Elemento Tabella[10];

Elemento riempiElemento(Dati d, Indirizzo i){
    Elemento e;

    e.matr=d.matr;
    e.CDL=d.CDL;
    strcpy(e.nome, i.nome);
    strcpy(e.cognome, i.cognome);
    strcpy(e.via, i.via);
    strcpy(e.citta, i.citta);
    e.CAP=i.CAP;

    return e;
}

int main() {
    Dati D;
    Indirizzo I;
    Elemento E;
    Tabella T;
    FILE *f1, *f2;
    int i, trovato, ins=0, totC;
    unsigned int C;

    f1=fopen("dati.txt", "r");
```

```
    f2=fopen("indirizzi.txt", "r");
    while (fscanf(f1,"%u%u", &D.matr, &D.CDL)>0) {
        trovato=0;
        rewind(f2);
        while(fscanf(f2, "%d %s %s %s %s %d", &I.matr, I.nome, I.cognome, I.via, I
            .citta, &I.CAP) == 6 && !trovato)
            if (I.matr==D.matr) {
                trovato=1;
                E = riempiElemento(D, I);
                T[ins] = E;
                ins++;
            }
    }
    fclose(f1);
    fclose(f2);

    printf("Inserire il corso C: ");
    scanf("%u", &C);
    totC=0;
    for(i=0; i<ins; i++){
        if(T[i].CDL==C)
            totC++;
    }
    printf("\n Iscritti al corso %u: %f \%\n", C, (float)totC * 100 / ins);

    f1=fopen("milano.txt", "w");
    for (i=0; i<ins; i++){
        if (strcmp("Milano", T[i].citta) == 0)
            fprintf(f1,"%s %s %u\n",T[i].nome,T[i].cognome,T[i].matr);
    }
    fclose(f1);
    return 0;
}
```

Soluzione dell'esercizio 9.5

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DIM 32

typedef struct {
    char name[DIM];
    int points;
} User;

int readPoints (char usersFile[], User results[], int maxDim, int minPoints){
    FILE * f; int logicDim = 0;

    f = fopen(usersFile, "rb");
    if (f == NULL)
        return -1;

    while(logicDim<maxDim && fread( &results[logicDim], sizeof(User), 1, f) > 0) {
        if (results[logicDim].points >= minPoints)
            logicDim++;
    }

    fclose(f);
```

```
    return logicDim;
}


int main() {
    User * V; int i, maxUtenti; int logicDim, minPoints;

    printf("Inserire numero massimo di utenti da leggere: ");
    scanf("%d", &maxUtenti);

    V = (User *) malloc(sizeof(User) * maxUtenti);
    if (V == NULL)
        return -1;

    printf("Inserire punteggio minimo: ");
    scanf("%d", &minPoints);
    logicDim = readPoints("punti.dat", V, maxUtenti, minPoints);
    if (logicDim < 0)
        return -1;

    for (i=0; i<logicDim; i++){
        if ((V[i].name[0] == 'M') && (V[i].name[1] == 'e')) {
            printf("L'utente %s ha %d punti.\n", V[i].name, V[i].points);
        }
    }

    free(V);
    return 0;
}
```

Soluzione dell'esercizio 9.6

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void stampa_uso(char *nome_prog) {
    printf("Errore nei parametri.\n");
    printf("Uso: %s -i <input> -o <output> -k <chiave> [-d]\n", nome_prog);
}

int main(int argc, char *argv[]) {
    char *file_in = NULL;
    char *file_out = NULL;
    int chiave = 0;
    int chiave_trovata = 0;
    int decifra = 0;

    int j;
    FILE *fin, *fout;
    int c;

    for (j = 1; j < argc; j++) {
        if (strcmp(argv[j], "-i") == 0) {
            if (j + 1 < argc) {
                file_in = argv[++j];
            }
            else {
                return 1;
            }
```

```
        }
        else if (strcmp(argv[j], "-o") == 0) {
            if (j + 1 < argc) {
                file_out = argv[++j];
            }
            else {
                return 1;
            }
        }
        else if (strcmp(argv[j], "-k") == 0) {
            if (j + 1 < argc) {
                chiave = atoi(argv[++j]);
                chiave_trovata = 1;
            } else {
                return 1;
            }
        }
        else if (strcmp(argv[j], "-d") == 0) {
            decifra = 1;
        }
    }

    if (file_in == NULL || file_out == NULL || chiave_trovata == 0) {
        stampa_uso(argv[0]);
        return 1;
    }

    fin = fopen(file_in, "r");
    if (fin == NULL) return 1;

    fout = fopen(file_out, "w");
    if (fout == NULL) { fclose(fin); return 1; }

    if (decifra) {
        chiave = -chiave;
    }

    while ((c = fgetc(fin)) != EOF) {

        // Caso Maiuscole: da 'A' (65) a 'Z' (90)
        if (c >= 'A' && c <= 'Z') {
            // Normalizziamo da 0 a 25 sottraendo 'A'
            // Aggiungiamo la chiave
            // Facciamo il modulo 26
            c = (c - 'A' + chiave) % 26;

            // Gestione modulo per numeri negativi (decifratura)
            if (c < 0) c += 26;

            // Riportiamo nel range ASCII sommando 'A'
            c += 'A';
        }

        // Caso Minuscole: da 'a' (97) a 'z' (122)
        else if (c >= 'a' && c <= 'z') {
            c = (c - 'a' + chiave) % 26;

            if (c < 0) c += 26;

            c += 'a';
        }
```

```
        /* Se non è né maiuscola né minuscola (es. numeri, punteggiatura),
           saltiamo gli if e scriviamo il carattere originale */

        fputc(c, fout);
    }

    printf("Operazione completata.\n");

    fclose(fin);
    fclose(fout);

    return 0;
}
```

Soluzione dell'esercizio 9.7

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *nome_file_db = "registro.txt";

// Prototipi
void stampa_uso(char *prog_name);
void aggiungi_studente(char *nome, char *cognome, int voto);
void cerca_studenti(int soglia_voto);

int main(int argc, char *argv[]) {
    int i;
    int azione_trovata = 0; // 0 = nessuna, 1 = add, 2 = find */

    int idx_add = -1;
    int idx_find = -1;

    for (i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-f") == 0) {
            if (i + 1 < argc) {
                nome_file_db = argv[i + 1]; // Sovrascrive il default globale
                i++; // Salta il prossimo elemento (il nome del file)
            } else {
                printf("Errore: Manca il nome del file dopo -f\n");
                return 1;
            }
        }
        else if (strcmp(argv[i], "-add") == 0) {
            azione_trovata = 1;
            idx_add = i;
        }
        else if (strcmp(argv[i], "-find") == 0) {
            azione_trovata = 2;
            idx_find = i;
        }
    }

    if (azione_trovata == 0) {
        stampa_uso(argv[0]);
        return 0;
    }

    if (azione_trovata == 1) { // Modalità ADD
```

```
        if (idx_add + 3 < argc) {
            char *n = argv[idx_add + 1];
            char *c = argv[idx_add + 2];
            int v = atoi(argv[idx_add + 3]);
            aggiungi_studente(n, c, v);
        } else {
            printf("Errore: -add richiede <nome> <cognome> <voto>\n");
        }
    }
    else if (azione_trovata == 2) { // Modalità FIND
        if (idx_find + 1 < argc) {
            int soglia = atoi(argv[idx_find + 1]);
            cerca_studenti(soglia);
        } else {
            printf("Errore: -find richiede <voto_minimo>\n");
        }
    }

    return 0;
}

void stampa_uso(char *prog_name) {
    printf("Gestore Registro Voti v1.0\n");
    printf("Uso: %s [-f filename] <azione>\n", prog_name);
    printf("Azioni:\n");
    printf("  -add <nome> <cognome> <voto>  : Aggiunge uno studente\n");
    printf("  -find <voto_min>              : Cerca studenti con voto >= soglia\n"
        );
}

void aggiungi_studente(char *nome, char *cognome, int voto) {
    FILE *fp;

    // Apertura in "a" (append): scrive in coda senza cancellare
    fp = fopen(nome_file_db, "a");
    if (fp == NULL) {
        printf("Errore critico: Impossibile scrivere su '%s'\n", nome_file_db);
        return;
    }

    fprintf(fp, "%s %s %d\n", nome, cognome, voto);

    printf(">> Studente aggiunto correttamente in '%s'.\n", nome_file_db);

    fclose(fp); // Chiude il file per salvare le modifiche
}

void cerca_studenti(int soglia_voto) {
    FILE *fp;
    char n[50], c[50];
    int v;
    // Variabile STATICA: Viene inizializzata solo una volta all'avvio del
        programma.
    // Mantiene il valore anche dopo che la funzione termina
    static int record_trovati = 0;

    fp = fopen(nome_file_db, "r");
    if (fp == NULL) {
        printf("Nessun database trovato (%s)\n", nome_file_db);
        return;
    }
```

```
    printf("Risultati (Voto >= %d)\n", soglia_voto);

    while (fscanf(fp, "%s %s %d", n, c, &v) == 3) {
        if (v >= soglia_voto) {
            printf("[%d] %s %s: %d\n", ++record_trovati, n, c, v);
        }
    }

    if (record_trovati == 0) {
        printf("Nessuno studente trovato con i criteri richiesti.\n");
    } else {
        printf("Totale record visualizzati: %d\n", record_trovati);
    }

    fclose(fp);
}
```

Soluzione dell'esercizio 9.8

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "lista.c"

#define MAX_AUTORE 24
#define MAX_TITOLO 56
#define MAX_VOLUMI 30

typedef struct {
    char autore[MAX_AUTORE];
    char titolo[MAX_TITOLO];
    int copie_possedute;
    int copie_prestito;
} Volume;


int main() {
    FILE *fp;
    Volume V[MAX_VOLUMI];
    int num_volumi = 0;
    int i;

    ListaDiElem L = NULL;
    ListaDiElem curr;

    int diff;
    int sum_possedute = 0;
    int sum_disponibili = 0;
    float rapporto;

    int max_val = -1;
    int riga_max = -1;
    int riga_corrente;

    fp = fopen("archivio.txt", "r");
    if (fp == NULL) {
        fprintf(stderr, "Errore: impossibile aprire 'archivio.txt'.\n");
        return 1;
    }
```

```
while (num_volumi < MAX_VOLUMI &&
       fscanf(fp, "%s %s %d %d",
              V[num_volumi].autore,
              V[num_volumi].titolo,
              &V[num_volumi].copie_possedute,
              &V[num_volumi].copie_prestito) == 4) {
    num_volumi++;
}
fclose(fp);

printf("--- CONTENUTO DEL VETTORE V ---\n");
for (i = 0; i < num_volumi; i++) {
    printf("Riga %d: %s - %s (Poss: %d, Prest: %d)\n",
           i + 1, V[i].autore, V[i].titolo, V[i].copie_possedute, V[i].
               copie_prestito);
}
printf("\n");


for (i = 0; i < num_volumi; i++) {
    diff = V[i].copie_possedute - V[i].copie_prestito;
    L = inserisciCoda(L, diff);
}

printf("--- CONTENUTO DELLA LISTA L (Copie Disponibili) ---\n");
stampaLista(L);
printf("\n");

for (i = 0; i < num_volumi; i++) {
    sum_possedute += V[i].copie_possedute;
}

curr = L;
while (curr != NULL) {
    sum_disponibili += curr->dato;
    curr = curr->prox;
}

if (sum_possedute > 0) {
    rapporto = (float)sum_disponibili / (float)sum_possedute;
    printf("--- STATISTICHE ---\n");
    printf("Totale Possedute (da V): %d\n", sum_possedute);
    printf("Totale Disponibili (da L): %d\n", sum_disponibili);
    printf("Rapporto: %.2f\n\n", rapporto);
}

curr = L;
riga_corrente = 1;

while (curr != NULL) {
    if (curr->dato > max_val) {
        max_val = curr->dato;
        riga_max = riga_corrente;
    }
    curr = curr->prox;
    riga_corrente++;
}

printf("--- RISULTATO ---\n");
if (riga_max != -1) {
    printf("Volume con piu' copie disponibili (%d) alla riga: %d\n", max_val,
        riga_max);
```

```
    } else {
        printf("Nessun dato disponibile.\n");
    }

    eliminaLista(L);

    return 0;
}
```